

Complete Notes on Regular Expression (Regex) Patterns

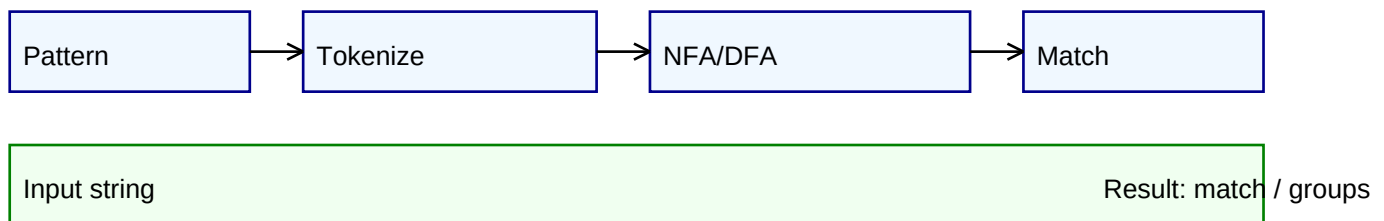
Overview

Regular Expressions (regex) are declarative patterns used to match, search, and transform text. They power validation (emails, phone numbers), parsing (logs, data extraction), and text processing (find/replace, tokenization).

Regex Engine (Conceptual)

Most modern engines are backtracking (PCRE-style). They tokenize the pattern, build an internal state machine (NFA/DFA-like), and attempt matches with backtracking when needed. This diagram is conceptual:

Regex engine conceptual flow



Core Building Blocks

Literals: Exact characters (e.g., cat).

Metacharacters: Characters with special meaning.

.	any char (except newline in many engines)
^	start of string/line
\$	end of string/line
*	0 or more
+	1 or more
?	0 or 1 (or quantifier modifier)
	alternation
()	group (capture)
[]	character class
{}	quantifier
\	escape special meaning

Character Classes

[abc]	one of a, b, c
[^abc]	not a, b, or c
[a-z]	lowercase letters
[A-Z]	uppercase letters
[0-9]	digits
[\w\W]	word / non-word (letters, digits, underscore)
[\d\D]	digit / non-digit
[\s\S]	whitespace / non-whitespace

Quantifiers

*	0 or more	+	1 or more
?	0 or 1	{n}	exactly n
{n,}	at least n	{n,m}	between n and m (inclusive)

Greedy by default; add ? for non-greedy (e.g., .*?)

Anchors & Boundaries

^, \$	start / end of string (or line if multiline)
-------	--

`\b, \B` word boundary / non-boundary
`\A, \Z` absolute start / end (in some engines)

Grouping, Capturing, and Backreferences

`( ... )` capturing group
`(?: ... )` non-capturing group
`(?P<name> ... )` named group (Python-style)
`\1, \2` numeric backreferences
`\k<name>` named backreference (PCRE/Python)

Lookarounds (Zero-width assertions)

`(?=...)` positive lookahead `(?!...)` negative lookahead
`(?<=...)` positive lookbehind `(?<!...)` negative lookbehind
Zero-width: they assert context without consuming characters.

Flags / Modifiers

`/i` case-insensitive `/m` multiline (`^` and `$` match line boundaries)
`/s` dot-all (dot matches newline) `/x` verbose mode (ignore whitespace, allow comments)
`/u` unicode mode `/g` global (JavaScript); use `findall` in Python

Common Patterns (Practical)

Email (simplified):
`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`

IPv4 (basic):
`^(\d{1,3}\.){3}\d{1,3}$`

ISO date (YYYY-MM-DD):
`^\d{4}-\d{2}-\d{2}$`

URL (very basic):
`^https?://[^\s]+$`

Tips & Best Practices

Keep patterns readable; use verbose/extended mode where available. Add tests for edge cases. Prefer explicit patterns over overly permissive ones. When validation matters, combine regex checks with business rules (e.g., DNS check for emails).

Performance Considerations

Avoid catastrophic backtracking by limiting nested quantifiers with wide matches like `'(.*)+'`. Prefer atomic groups or possessive quantifiers where supported (e.g., `.+*` in some engines).

Examples with Explanations

Extracting hashtags:

Pattern: `/#[A-Za-z0-9_]+\b/`
Text: "Having fun at #HyderabadTech and #Regex101!"
Groups: 1 -> HyderabadTech, Regex101

Validating a simple Indian mobile number (+91 optional):

`^\+?91?\s?-?\d{10}$`

Advanced Topics

Atomic groups: `(?>...)` prevent backtracking inside the group (PCRE).

Possessive quantifiers: `a++` or `.++` keep the match and disallow backtracking (Java/PCRE).

Conditional groups: `(?(?=cond)yes|no)` apply alternation based on an assertion.

Unicode classes: `\p{L}` (letters), `\p{N}` (numbers), `\p{Han}`, etc., when supported.

Testing & Debugging

Use unit tests with representative fixtures. Enable verbose mode. In many tools/IDEs you can visualize group boundaries and backtracking. Break complex patterns into smaller ones and combine programmatically.

Conclusion

Regex is powerful and concise. With careful design, readable patterns, and tests, you can solve most text tasks reliably.